

Manual: Engenharia de Software

Ciclos de Vida (Preditivos e RUP), Metodologias Ageis (Scrum, Kanban, XP), Engenharia de Requisitos, Modelagem UML, Arquiteturas, Padroes GoF e Testes

CONCURSO: SEFAZ-BA 2026

Cargo: Auditor Fiscal (TI) & Agente de Tributos Estaduais (Geral)

Foco de Bancas: FGV e Cebraspe

Este material foi consolidado com base na bibliografia recomendada e no edital.

1. Modelos de Ciclo de Vida de Software

O ciclo de vida do software descreve as fases pelas quais o sistema passa, desde a concepção até o descarte. Os modelos de ciclo de vida definem a ordem e o rito de execução dessas fases.

Modelos Clássicos (Preditivos) e Iterativos

- **Modelo Cascata (Waterfall):**

Modelo sequencial e rígido. Cada fase (Requisitos, Projeto, Codificação, Testes, Manutenção) deve ser totalmente concluída antes do início da próxima. Dificulta alterações de requisitos ao longo do projeto.

- **Modelo Espiral (Barry Boehm):**

Modelo evolucionário estruturado em ciclos repetitivos (espirais), com foco central na ANÁLISE DE RISCOS em cada volta da espiral.

- **RUP (Rational Unified Process):**

Processo iterativo orientado a Casos de Uso e centrado na arquitetura. Divide-se em 4 fases sequenciais: 1) Iniciação (Inception - define escopo inicial); 2) Elaboração (Elaboration - projeta a arquitetura e mitiga riscos); 3) Construção (Construction - desenvolve o sistema); 4) Transição (Transition - implantação e testes no cliente).

RUP E AS DISCIPLINAS

Atenção para as provas: no RUP, o trabalho é estruturado em Disciplinas (ex: Requisitos, Análise e Projeto, Implementação, Testes) que ocorrem simultaneamente, com intensidades variadas, ao longo de todas as 4 Fases do projeto.

2. Metodologias Ageis: Scrum, Kanban e XP

As metodologias ageis priorizam a adaptabilidade, entregas frequentes de software funcionando e colaboracao com o cliente, conforme o Manifesto Agil de 2001.

O Framework Scrum

Scrum e um framework leve projetado para ajudar equipes a gerar valor por meio de solucoes adaptativas para problemas complexos. Seus pilares sao: Transparencia, Inspecao e Adaptacao.

- **Papeis no Scrum:**

1) Product Owner (PO - dono do produto, representa os clientes e define a prioridade do Product Backlog); 2) Scrum Master (SM - lider servidor que garante a aplicacao do Scrum e remove impedimentos); 3) Developers (Equipe de Desenvolvimento - constroi o incremento).

- **Eventos do Scrum:**

Sprint (ciclo de desenvolvimento de 1 a 4 semanas); Sprint Planning (planejamento da meta); Daily Scrum (reuniao diaria de 15 minutos); Sprint Review (demonstracao do incremento concluido); Sprint Retrospective (analise de melhoria de processos da equipe).

- **Artefatos do Scrum:**

Product Backlog (lista ordenada de tudo que e necessario no produto); Sprint Backlog (tarefas selecionadas para a Sprint atual); Incremento (soma de itens concluidos na Sprint).

Kanban e XP (Extreme Programming)

- **Kanban:**

Metodo visual focado no gerenciamento do fluxo de trabalho. Utiliza quadros de colunas ('A Fazer', 'Em Execucao', 'Concluido') e limita o Trabalho em Progresso (limite de WIP) para evitar gargalos.

- **XP (Extreme Programming):**

Metodologia agil voltada para engenharia de software com foco em qualidade tecnica. Praticas principais: Programacao em Par (Pair Programming), Test-Driven Development (TDD), Refatoracao e Integracao Continua.

O SCRUM MASTER NAO E CHEFE

O Scrum Master nao e um gerente de projetos tradicional ou chefe da equipe. Ele e um facilitador e lider servidor. Nao atribui tarefas aos desenvolvedores; a equipe de desenvolvimento e auto-organizavel e define como executar o trabalho.

3. Engenharia de Requisitos

A engenharia de requisitos engloba as atividades de descoberta, análise, especificação e gerenciamento das necessidades dos usuários do sistema.

Classificação dos Requisitos

- **Requisitos Funcionais (RF):**
Descrevem as funções e serviços que o sistema deve fornecer (o que o sistema deve fazer). Ex: 'O sistema deve permitir a emissão de Nota Fiscal Eletrônica'.
- **Requisitos Não Funcionais (RNF):**
Descrevem as restrições, qualidades ou propriedades sobre os serviços fornecidos (como o sistema deve fazer). Abrangem desempenho, segurança, usabilidade, confiabilidade e portabilidade. Ex: 'O sistema deve responder as consultas em menos de 2 segundos' (Desempenho).
- **Regras de Negócio (RN):**
Diretrizes, normas ou políticas da organização que determinam como as ações ocorrem. Não são requisitos de software diretos, mas guiam a criação deles. Ex: 'O desconto máximo para clientes VIP é de 15%'.

Processo de Requisitos

- **Elicitação e Análise:**
Descoberta de requisitos através de entrevistas, brainstorming, prototipação e observação.
- **Especificação:**
Documentação dos requisitos em formato legível (Documento de Requisitos, Histórias de Usuário ou Casos de Uso).
- **Validação:**
Verificação com o cliente de que os requisitos especificados são realmente o que eles desejam.

HISTÓRIAS DE USUÁRIO (USER STORIES)

Em metodologias ágeis, os requisitos são documentados como Histórias de Usuário sob o formato: 'Como um [ator], eu quero [ação], para que eu possa [benefício]'. São acompanhadas por Critérios de Aceitação.

4. Modelagem UML: Diagramas Estruturais

A UML (Unified Modeling Language) é a notação padrão para modelagem orientada a objetos de sistemas de software, dividida em diagramas estruturais e comportamentais.

Diagrama de Classes (O mais Cobrado)

Representa a estrutura estática do sistema, mostrando as classes, seus atributos, métodos e relacionamentos:

- **Associação Simples:**
Relacionamento básico de conexão estrutural entre duas classes. Ex: Cliente compra Produto.
- **Agregação (Relação Todo-Parte fraca - Losango Vazio):**
Indica que a classe parte pode existir independentemente da classe todo. Ex: Computador possui Monitores externos (se o computador for destruído, os monitores continuam existindo).
- **Composição (Relação Todo-Parte forte - Losango Preenchido):**
Indica dependência física ou existencial da parte em relação ao todo. Se o todo for destruído, as partes também são destruídas. Ex: Nota Fiscal possui Itens da Nota Fiscal.
- **Generalização / Especialização (Herança - Seta Vazia):**
Indica herança de atributos e métodos de uma superclasse por uma subclasse. Ex: Pessoa Física herda de Pessoa.

AGREGAÇÃO VS. COMPOSIÇÃO NO DIAGRAMA

O losango de Agregação é sempre VAZIO (transparente) e indica independência. O losango de Composição é sempre PREENCHIDO (preto) e indica que a parte não tem vida própria sem o todo. Não erre essa representação visual na prova!

5. Modelagem UML: Diagramas Comportamentais

Os diagramas comportamentais mostram a dinamica do sistema, a interacao entre os objetos e a mudanca de estado ao longo do tempo.

Diagrama de Casos de Uso

Mostra as interacoes entre Atores (usuarios ou sistemas externos) e os Casos de Uso (funcoes do sistema). Os relacionamentos principais entre Casos de Uso sao:

- **Include (Inclusao - Seta pontilhada com <<include>>):**
Indica que o caso de uso de origem exige obrigatoriamente a execucao do caso de uso de destino. Ex: 'Sacar Dinheiro' inclui 'Validar Senha'.
- **Extend (Extensao - Seta pontilhada com <<extend>>):**
Indica comportamento opcional ou condicional que estende o caso de uso de origem. Ex: 'Efetuar Pagamento' e estendido por 'Exibir Alerta de Saldo Baixo' (condicionalmente).

Diagrama de Sequencia

Diagrama de interacao focado no envio de mensagens ordenadas temporalmente entre linhas de vida (objetos) para realizar uma operacao.

Diagrama de Atividades

Representa o fluxo de controle de um processo de negocio ou algoritmo, assemelhando-se a um fluxograma.

DIRECAO DA SETA DE RELACIONAMENTOS

No relacionamento <<include>>, a seta aponta do caso de uso base para o caso incluido. No relacionamento <<extend>>, a seta aponta do caso extensor (opcional) para o caso de uso base. Essa direcao e alvo constante de pegadinhas de bancas.

6. Arquiteturas de Software: MVC, Microservicos e SOA

A arquitetura de software define a estrutura organizacional do sistema, seus componentes, relacionamentos e principios de design.

MVC (Model-View-Controller)

Padrao de arquitetura que divide a aplicacao em tres camadas interconectadas para separar a representacao da informacao do controle do usuario:

- **Model (Modelo):**
Gere os dados, as regras de negocio, a persistencia e o estado do sistema.
- **View (Visao):**
Representacao visual dos dados para o usuario (interface grafica, telas HTML/CSS).
- **Controller (Controlador):**
Intercepta a entrada do usuario na View, converte as acoes em comandos para o Model e atualiza a View correspondente.

Microservicos vs. Monolitico

- **Monolitico:**
O sistema e construido como uma unica unidade autônoma de implantacao. Facil de testar inicialmente, mas complexo de escalar e manter a medida que cresce.
- **Microservicos:**
O sistema e decomposto em um conjunto de servicos pequenos, independentes, autonomos e altamente focados. Cada servico roda seu proprio processo e se comunica por APIs (REST). Facilita escalabilidade e tolerancia a falhas.

SOA - SERVICE-ORIENTED ARCHITECTURE

SOA e um estilo de arquitetura de software baseado no fornecimento de servicos de negocio reutilizaveis. Diferente de microservicos (que visam desacoplamento fisico maximo de implantacao), SOA foca no compartilhamento e integracao de sistemas legados atraves de um barramento de servicos (ESB).

7. Padroes de Projeto GoF (Gang of Four)

Os padroes de projeto sao solucoes consolidadas para problemas comuns recorrentes no desenvolvimento orientados a objetos. Dividem-se em tres categorias principais:

1. Padroes Criacionais (Criacao de Objetos)

- **Singleton:**
Garante que uma classe tenha apenas uma unica instancia na memoria de toda a aplicacao e fornece um ponto de acesso global a ela.
- **Factory Method / Abstract Factory:**
Factory Method define uma interface para criar um objeto, mas deixa as subclasses decidirem qual classe instanciar. Abstract Factory cria familias de objetos relacionados sem especificar suas classes concretas.

2. Padroes Estruturais (Composicao de Classes/Objetos)

- **Adapter:**
Converte a interface de uma classe em outra interface esperada pelo cliente, permitindo a cooperacao de interfaces incompativeis. Ex: plugues e tomadas de padroes diferentes.
- **Facade (Fachada):**
Fornece uma interface simplificada para um subsistema complexo.
- **Decorator (Decorador):**
Adiciona responsabilidades extras a um objeto de forma dinamica (em tempo de execucao) sem usar heranca.

3. Padroes Comportamentais (Comunicacao entre Objetos)

- **Observer (Observador):**
Define uma dependencia um-para-muitos, notificando automaticamente todos os observadores de qualquer mudanca de estado no objeto observado.
- **Strategy (Estrategia):**
Define uma familia de algoritmos, encapsula cada um e os torna intercambiaveis em tempo de execucao.

STRATEGY VS. STATE

Ambos possuem diagramas UML parecidos, mas objetivos distintos: o Strategy permite alterar algoritmos intercambiáveis de forma explícita pelo cliente. O State permite alterar o comportamento de um objeto conforme o seu estado interno muda de forma transparente.

8. Qualidade e Testes de Software: Caixa Preta vs. Branca

Os testes de software garantem que o sistema atenda aos requisitos especificados e esteja livre de falhas críticas.

Níveis de Testes

- **Teste Unitário (de Unidade):**
Testa a menor unidade isolada de código (funções, métodos ou classes). Executado pelos próprios desenvolvedores.
- **Teste de Integração:**
Avalia a interação e comunicação entre diferentes módulos ou unidades integradas do sistema.
- **Teste de Sistema (ou de Sistema Completo):**
Avalia o comportamento global do sistema em relação aos requisitos funcionais e não funcionais especificados.
- **Teste de Aceitação (UAT):**
Validação final realizada pelos usuários finais para verificar se o sistema atende às suas necessidades de negócio antes de ir para produção.

Técnicas de Testes de Software

- **Caixa Preta (Funcional):**
O testador não conhece o código fonte interno. Os testes são focados nas entradas e saídas em relação à especificação funcional do sistema. Ex: teste de interface de login.
- **Caixa Branca (Estrutural):**
O testador conhece o código fonte interno. Os testes avaliam caminhos lógicos, condicionais e estruturas internas do código. Ex: garantir que todas as linhas de um algoritmo 'IF/ELSE' sejam executadas (cobertura de código).

CI/CD - INTEGRAÇÃO E IMPLANTAÇÃO CONTÍNUAS

A *Integração Contínua (CI)* é a prática de integrar mudanças de código ao repositório principal com frequência, rodando testes automáticos. A *Implantação Contínua (CD)* automatiza o deploy seguro dessas mudanças nos ambientes de produção.

9. As 10 Grandes Pegadinhas de Engenharia de Software

As bancas FGV e Cebraspe exploram pegadinhas conceituais sobre engenharia de software de forma frequente. Atente-se aos seguintes pontos:

- **1. Cascata nao permite retorno:**

No modelo cascata classico, nao ha caminhos de retorno (feedback loops) entre as fases. Uma fase so comeca se a anterior estiver 100% pronta.

- **2. Fases do RUP vs. Disciplinas:**

As fases do RUP sao Incepcao, Elaboracao, Construcao e Transicao. Disciplinas sao Requisitos, Analise, etc. As bancas trocam os nomes de fases por disciplinas.

- **3. Product Owner define prioridade, nao o Scrum Master:**

O PO e o unico responsavel por priorizar e ordenar os itens do Product Backlog. O SM apenas facilita os ritos e remove impedimentos.

- **4. Criterio de Aceitacao nao e Historia de Usuario:**

As Historias de Usuario definem o requisito do ponto de vista do ator. Os Criterios de Aceitacao definem o que deve ser validado para a historia ser considerada concluida (Definition of Done).

- **5. Agregacao vs. Composicao no UML:**

Composicao (losango preto) indica dependencia vital da parte. Agregacao (losango vazio) indica ciclo de vida independente. A banca adora trocar os losangos no Diagrama de Classes.

- **6. Include e Obrigatorio, Extend e Condicional:**

No UML de Casos de Uso, include e de execucao obrigatoria do caso secundario. O extend e de execucao condicional/opcional.

- **7. Factory Method vs. Abstract Factory:**

Factory Method usa heranca (um metodo criador em subclasse). Abstract Factory usa composicao (objeto fabricante contendo metodos para fabricar familias de objetos).

- **8. Decorator nao usa Heranca estrutural:**

O Decorator estende o comportamento de forma dinamica atraves de encapsulamento de objetos, evitando a criacao de subclasses estaticas complexas via heranca.

- **9. Caixa Preta nao avaliaCodigo Fonte:**

Questoes que dizem que testes de caixa preta garantem a cobertura de caminhos do codigo estao incorretas. Caixa preta avalia apenas entradas e saidas funcionais.

- **10. Risco de TDD:**

O Test-Driven Development (TDD) exige escrever o caso de teste ANTES de escrever o codigo de producao correspondente (Ciclo Red-Green-Refactor).

FICHA DE REVISAO GERAL

Foque os estudos finais nos ritos e papeis do Scrum (PO, SM, Devs), nos relacionamentos de include/extend em Casos de Uso, nos padroes GoF (Singleton, Factory, Strategy) e nos conceitos de testes (Caixa Preta vs. Branca).